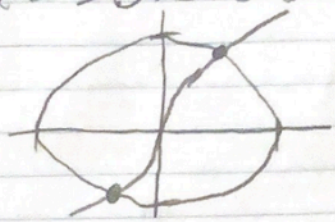
# Homework 6

Alex Opermann

$x_1^3 - x_2^3 + x_1 = 0$ and $x_1^2 + x_2^2 - 1 = 0$
when $x = \pm 0.508$ and $y = \pm 0.861$
graph:



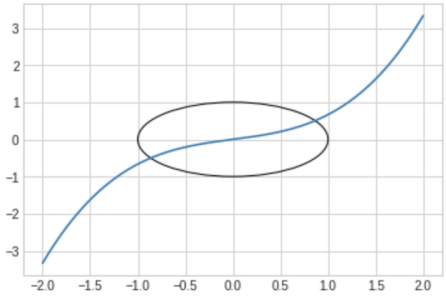
2. $J = \begin{vmatrix} \partial f_1/\partial x_1 & \partial f_1/\partial x_2 \\ \partial f_2/\partial x_1 & \partial f_2/\partial x_2 \end{vmatrix} = \begin{vmatrix} 3x_1^2+1 & -3x_2^2 \\ 2x_1 & 2x_2 \end{vmatrix} = 6x_1^2x_2 + 2x_1 - 6x_2^2x_1$

4. I chose $x_1$ to be 1 and $x_2$ to be 0. It failed because the Jacobian isn't invertible.

[68]:
```
import math
import numpy as np
def f1(x,y):
    return x**3-y**3+x
def f2(x,y):
    return x**2+y**2-1
```

[69]:
```
%matplotlib inline
import matplotlib.pyplot as plt
plt.style.use('seaborn-whitegrid')
figure, axes = plt.subplots()
xr=np.arange(-2,2,0.001)
yr=(xr**3+xr)**1/3
func=f1(xr,yr)
axes.plot(xr,yr)
cir=plt.Circle((0,0),1,fill=False)
axes.add_artist(cir)
```

[69]:
```
<matplotlib.patches.Circle at 0x7fdd52c9f370>
```




[88]:
```
def jacobian(x,y):
    #calculated manually in problem #2
    l1=[3*x**2+1,-3*y**2]
    l2=[2*x,2*y]
    return [l1,l2]
def newtonsMethod(x1,x2):
    x=[x1,x2]
    f=[f1(x1,x2),f2(x1,x2)]
    h = np.linalg.solve(jacobian(x1,x2),f)
    count = 0
    while abs(h[0]) >= 0.0001 or abs(h[1]) >= 0.0001:
        if jacobian(x1,x2) == None:
            return None
        f=[f1(x1,x2),f2(x1,x2)]
        h = np.linalg.solve(jacobian(x1,x2),f)
        x = x - h
        x1=x[0]
        x2=x[1]
        count = count + 1
        if count > 1000:
            return None
    return x
ans=newtonsMethod(1,1)
print("%f %f"%(ans[0],ans[1]))
```

```
0.507992 0.861362
```